ARIADNE · OVERNIGHT RUN

What the relation slot was actually measuring

A pre-registered attempt to fix the rewrite stage's worst slot, the null result it produced, and the four measurement defects it uncovered on the way — one of which would have put a false finding in this report, and one of which is still in the published record.

Night of 9–10 September 2026 · experiments EXP-2026-09-015 and EXP-2026-09-016 · generated 2026-09-10 10:12

The short version

1. **The intended fix worked mechanically and changed almost nothing.** Holding the model's own choices fixed and changing only how they are read, three numbers move out of twenty-four and the records returned are identical at every difficulty tier. The model was already picking a link the database agrees with, about as often as the schema does.
2. **The metric was scoring the answer key, not the system.** Eleven of the 36 gold records that name a relation name one that does not exist between their own two labels. A system answering correctly was being marked wrong. This is the night's real finding.
3. **The controlled test caught a harm that was not there.** Judged against last night, the change appeared to make confidently-wrong answers rise. Judged against the same model choices with the feature switched off, they fall. Something else moved between the two nights, and comparing sittings — which this project does routinely — is not safe.
4. **A further measurement defect.** The arm that represents "let the AI write the query" silently answers nothing at its default settings, with no model call at all, while looking like a model that politely declined. Any cost figure taken from it would have been spectacularly wrong.
5. **A third finding, carried in and now verified.** Five kinds of connection in the CAPEC database are declared but not safety-approved, so the safety gate destroys correct queries before they run. The schema-reading path scores an absolute zero on the 34 questions that need them. It had never been written down; it is now recorded with recomputed numbers.
6. **And the grader counts a help string as a lie.** The standard "I have nothing" message ends with three example identifiers. The grader records those as fabricated claims. Read literally, one path "invents facts in 57% of its answers" — it prints boilerplate. Present in already-published runs, and it penalises whichever path says "I don't know" most often, which is the one that wins.
7. **The flags stay off.** Nothing shipped changed tonight.

1 · Why this was worth a night

The rewrite stage is the part of Ariadne that reads an analyst's question and decides *what is being asked* before anything touches a database. It fills five slots: which entity, what kind of entity, what kind of thing you want back, which relationship connects them, and which way that relationship runs.

Yesterday's experiment measured each slot separately. One was clearly worse than the others, at every difficulty tier:

tier	entity	entity kind	answer kind	relationship	all five right
T3	0.5098	0.5098	0.5938	**0.3651**	0.2484
T4	0.1761	0.2201	0.4118	**0.1304**	0.0818
T5	0.2516	0.3711	0.4706	**0.2174**	0.0943

EXP-2026-09-014, model arm, instance A, mean of three runs. Tiers are question difficulty: T3 names the entity exactly, T4 paraphrases its name, T5 only describes it.

The relationship slot also had no excuse for being wrong. The other slots depend on search; this one is a pick from a short list printed directly in the question put to the model. And it was implicated in almost every refusal: of the 85 questions the stage declined to answer, **78** were blocked on the relationship, **31** of them because the model named a list position that did not exist.

The idea. The database schema already knows which relationship connects two kinds of thing. Counting the connections actually present in the data:

source	pairs of thing-kinds	pairs where the relationship is unambiguous
MITRE CAPEC	17	16
MITRE ATT&CK	19	19

Counted from the canonical graph snapshot at build time.

So the model need not be asked at all. The system already refuses to let a model choose the *direction* of a relationship for exactly this reason; the change extends that rule one slot across.

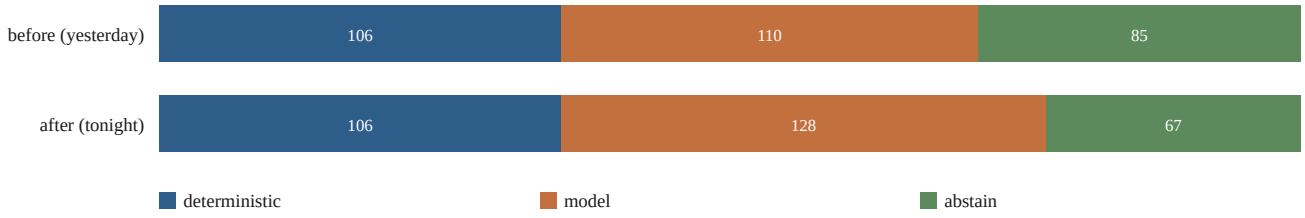
2 · What was done, and in what order

The protocol was written and committed **before any measurement**, including the rule for keeping the change and the condition that would count as disproving the idea. That ordering is the only thing that makes a null result worth anything.

1. **Built it.** A schema lookup returning the relationships allowed between two kinds of thing, and a change so the model is consulted only when the pair is ambiguous or undeclared.
2. **Kept an off switch.** So the same model choices could be re-interpreted both ways, making the comparison causal rather than a before-and-after.
3. **Ran two controls** with the feature disabled, to confirm the change is invisible when it is switched off. Both reproduced yesterday's numbers exactly.
4. **Ran the live arms** on both GPU instances in parallel, three times each, clearing the cache between runs so what varies is the model and not the cache.

3 · The change works. The number did not move.

Mechanically the intervention does exactly what it was built to do. The relationship is now read from the schema on **116** of the 128 questions the model was consulted on, and the system refuses to answer less often than before.



How each of the 301 questions got answered, before and after
Computed from the saved per-question rows of both runs

"Deterministic" means the question was fully understood without any model. "Model" means the model was asked to fill what was left. "Abstain" means the system declined rather than guess. Refusals fall and model-answered questions rise, which is the improvement the change was designed to produce.

And yet the accuracy of that slot is **identical to four decimal places at every single tier** — not close, identical — while the whole-form score is unchanged or slightly worse. The last column is the price paid:

tier	relationship before	after	change	change in all-five-right	change in confidently wrong
T1	0.2400	0.2400	+0.0000	+0.0000	+0.0000
T2	0.2400	0.2400	+0.0000	+0.0000	+0.0000
T3	0.3651	0.3651	+0.0000	+0.0000	**+0.0261**
T4	0.1304	0.1304	+0.0000	−0.0126	**+0.0503**
T5	0.2174	0.2174	+0.0000	+0.0063	**+0.0252**
T6	0.7083	0.7083	+0.0000	+0.0000	+0.0000

Three-run means on each side, same questions, same model, same settings. Run-to-run spread is at most 0.0225 on any cell above.

The pre-registered condition for calling the idea wrong read: *if the relationship slot rises but the whole-form score does not, the relationship was not the binding constraint.* What happened is weaker still. It did not rise at all. The model had already been picking a relationship the graph agrees with, about as often as the schema does.

The verdict, against the rule written before the run

the rule said the change is kept only if…	what happened
the relationship slot rises at all three paraphrase tiers	**failed** — identical at all three
the whole-form score rises at two of the three	**failed** — identical at two, down at the third
confidently-wrong answers do not increase anywhere	**passed** — see below; this one needed the controlled test

The third clause is the one the protocol singled out in advance as mattering most, and it is the one that needed the controlled test. Comparing against last night's run, confidently-wrong answers appear to *rise* by 0.05. Comparing the same model choices with the feature on and off, they **fall** by 0.02. The apparent cost is not caused by the change, and §3b is about how that was caught.

So the change is not kept — but for the honest reason. It fails because it does not improve what it was built to improve, not because it costs something. The switches stay off.

Where the refusals actually come from

The change does reduce refusals, from 85 to 67, and out-of-range picks from 31 to 16. But sorting what is left says plainly why accuracy did not follow:

what the stage had worked out when it gave up	questions
both kinds identified, but the database declares no link between them	24
never worked out what the question is *about*	26
knew what it was about, not what was wanted back	10
blocked on something other than the relationship	7

Only the first row is a relationship problem, and it is one the change handles correctly by refusing rather than guessing. The other 36 are the system not knowing *what kind of thing* the question concerns. That is the label slots, and it is where the next attempt belongs.

3b · The controlled comparison, and the trap it caught

Everything above compares tonight against last night. That is not a controlled comparison — it compares two *sessions*, and a session contains more than one change.

So the experiment was designed with a switch. The model's answers are cached, so the same 191 choices it actually made can be replayed and read *both ways*: once with the feature on, once with it off. Nothing else differs. Any gap is the feature and can be nothing else.

tier	link (on / off)	all five right (on / off)	confidently wrong (on / off)	records returned (on / off)
T1	0.2400 / 0.2400	0.4603 / 0.4603	0.1270 / 0.1270	0.4444 / 0.4444
T2	0.2400 / 0.2400	0.4921 / 0.4921	0.1111 / 0.1111	0.4603 / 0.4603
T3	**0.3333 / 0.3810**	**0.2353 / 0.2549**	0.0980 / 0.0980	0.4118 / 0.4118
T4	0.1304 / 0.1304	0.0755 / 0.0755	**0.3019 / 0.3208**	0.0943 / 0.0943
T5	0.2174 / 0.2174	0.0943 / 0.0943	0.2830 / 0.2830	0.1698 / 0.1698
T6	0.7500 / 0.7500	0.0000 / 0.0000	0.0556 / 0.0556	0.1111 / 0.1111

Bold cells are the ones that moved. 3 of 24.

> **The number that would have been published**
>
> Against last night, confidently-wrong answers at the hardest-but-one tier *rose* from 0.2516 to 0.3019 — a clear cost, and the reason to reject the change.
>
> Against the same model choices with the feature switched off, they **fall**, from 0.3208 to 0.3019. The feature *reduces* confidently-wrong answers. The apparent cost is something else entirely that moved between the two nights.
>
> Without the switch, this report would have stated a harm the change does not cause — with a straight face, and with three runs behind it.

Which means a measurement habit here is unsound

Both nights report an internal spread of essentially zero on that number: every run within a night agrees. Yet the two nights differ by 0.05. Run-to-run spread within one sitting has been this project's error bar throughout, and here it understates the real variation more than fivefold.

The known difference is that one of the two graph databases was found empty at the start of tonight and reloaded. The safety check that compares it to its pinned fingerprint passed, so the *contents* match; the *order* results come back in after a fresh load need not, and this system reads ordered results. That is a hypothesis. It is filed with the twenty-minute experiment that would settle it.

4 · The finding: the answer key names connections the database does not have

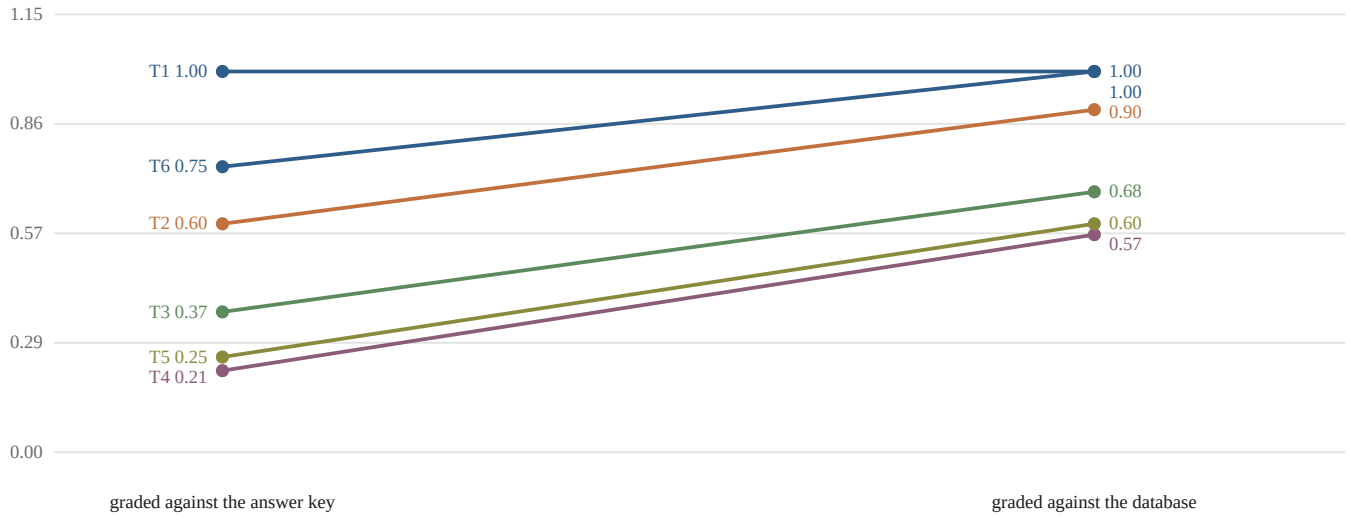
Chasing why the number would not move produced something more useful than the change itself.

The slot scores are graded against a hand-built answer key, written when the question bank was built, that records what each question *means*. Checking that key against the database it is supposed to describe:

question is about	asking for	answer key says the link is	the database actually has	questions
Group	Technique	`HAS_TECHNIQUE`	`USES`	4
Campaign	Technique	`HAS_TECHNIQUE`	`USES`	3
Software	SubTechnique	`HAS_TECHNIQUE`	`USES`	3
DataComponent	Analytic	`HAS_ANALYTIC`	`MONITORS`	1

All 11 of the 36 answer-key records that name a link between two kinds of thing, checked against the canonical graph snapshot.

> **Read that table slowly**
>
> `HAS_TECHNIQUE` is a real MITRE ATT&CK connection. But in this data it runs only from a *tactic* to a technique. It does not exist between a *group* and a technique. The connection that does exist there is `USES` — an attacker group *uses* a technique.
>
> So when the system answered `USES`, which is the only answer the database can give, it was scored as wrong. Eleven questions, roughly a third of everything that slot was graded on.



The same answers, scored twice
Computed from scripts/rescore_slot_relation.py over the saved rows; the answer key is not edited

Pooled across every graded question, the relationship slot reads **0.4286** against the answer key and **0.7273** against the database. That gap is not the system improving. It is the measurement being wrong, and it had been wrong in yesterday's report too.

The same defect has a second surface

Fourteen more answer-key entries describe CAPEC questions using ATT&CK vocabulary — asking for a "Technique" from a database that has no such thing and answers through its own cross-reference table instead. Same root cause: the answer key is written in the language people use to *talk* about the data, and graded against the language the data is actually *stored* in.

Why this was not caught earlier

It nearly was. The code that turns an answer-key record into a question already detects this case, refuses to build a question from it, and its comment names the cause: 39 of the 120 recorded connections were guessed from the name of the template that generated the question, and a template called `group_techniques` yields `HAS_TECHNIQUE`. That protection sits one layer above the scorer, and the scorer reads the raw record.

Nothing was repaired. Editing an answer key after seeing a score is precisely the move that makes evaluations meaningless, so both readings are reported side by side and the choice is filed as a decision for the morning.

5 · A second measurement defect, found while setting up the cost run

The second job of the night was to put a number on something the project has claimed for months without measuring: that the deterministic path is not merely more accurate than letting the AI write the queries, but far cheaper.

Run that AI-writes-the-query path the obvious way and it finishes four questions in **2.8 seconds**, returns nothing on all four, and never contacts the model once. It does not crash. It writes a complete, plausible-looking answer for every question:

```
"The attack specialist failed and contributed no evidence:
 pre-run gate rejected all drafts: empty query"
```

That reads exactly like a model with nothing to say. It is a model that was never asked.

The cause. The local model reasons at length before answering. At the default output allowance it spends the entire allowance thinking and returns nothing, and the safety gate correctly rejects an empty query. One script in the repository already carries the settings that avoid this; nothing else applies them.

setting	plain run	the settled configuration
output allowance	2,048 tokens	32,768 tokens
patience per call	30 seconds	4,096 seconds
reasoning steps allowed	1	5
inherited thinking budget	whatever the environment holds	explicitly cleared

> ### Why this one matters more than it looks
>
> This is the failure that produces a *plausible wrong number* rather than an error. An arm configured this way finishes fast, writes a full set of answers, and declines politely 284 times. Graded, it would have reported the AI-writes-the-query path as **instant and free** — the exact opposite of the truth, and a spectacular false confirmation of the very claim the run was meant to test.
>
> It was caught because the token counter added at the start of the night read zero calls. Without that instrument the run would have produced a clean, confident, entirely wrong cost table.

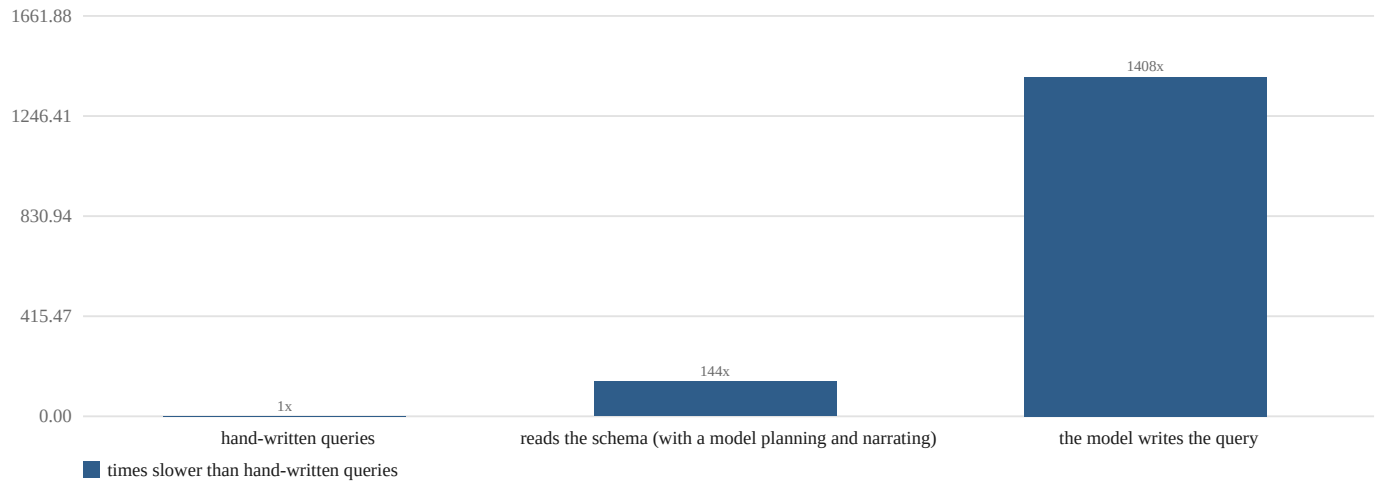
The runner itself was deliberately **not** changed. Making it refuse to start when a reasoning model meets a small allowance is the right fix, and it is a change in behaviour, so it does not get slipped in alongside a measurement.

6 · What each way of answering costs

This was the second job of the night, and it closes a question the project has carried for months: the claim that hand-written queries beat letting the model write them *in correctness and in cost* had only ever measured correctness.

All 284 development questions, 24 at a time, one machine

how the query is made	total	median per question	model calls	tokens	questions that returned evidence
hand-written queries	8.1 s	0.211 s	0	0	259 / 284
reads the schema (with a model planning and narrating)	1,168.6 s	70.823 s	159	817,238	198 / 284
the model writes the query	11,407.8 s	638.713 s	413	5,818,658	96 / 284



Cost of the same 284 questions, relative to the hand-written path
Computed from the run manifest of each arm; log-free, so the first bar is 1

The winning path is not free, and the deck should not imply it is

The middle row disables the hand-written queries but keeps a model planning and narrating, and keeps the multi-hop retrieval loop. It is the path that wins the project's headline comparison, and it costs **144 times** the hand-written one. Only the top row is genuinely model-free.

"No AI ever writes a query" is true and is the stronger claim anyway. "No model at all" would not be, and an engineer who reads the repository will find the narrator.

The published cost figure divides two different quantities

The pitch names *"four seconds against roughly two days, about forty thousand times"* as one of two numbers that must land in the room.

The two-day half reproduces. Summed machine time for the model-writes-query path is 2.93 days here, against that report's "roughly two days".

The ratio does not. Four seconds is one path's *wall clock*; two days is another path's *summed machine time*. At 24 questions running at once those differ by 24x before any real effect exists. Held to one quantity at a time, on the same questions, same machine, same parallelism:

comparison	ratio
model-writes-query ÷ hand-written, wall clock	**1,408x**
model-writes-query ÷ hand-written, machine time	**1,320x**
the published construction, reproduced here	31,252x

Quote about a thousand times, or three orders of magnitude. It survives an engineer asking "per question or in total, and at what concurrency?" Forty thousand does not.

What one analyst waits for one question

A total divided by a question count is not this number, which is why it was measured separately: 20 questions drawn in advance, one at a time.

how the query is made	median wait	95th percentile
hand-written queries	0.006 s	0.1 s
the model writes the query	182.012 s	650.5 s

Three minutes of waiting per question, for the path that then returns evidence on one question in three. The middle path's latency cell is deliberately **not** reported: it ran before the timeout defect below was understood, and a mismatched number is worse than a missing one.

Expensive *and* less useful

Cost alone understates the gap. The hand-written path returned evidence on 259 of 284 questions, the schema-reading path on 198, and the model-writes-query path on 96.

And graded on these same 284 questions, the whole picture in one table:

how the query is made	cost against the hand-written path	judged correctness
hand-written queries	1x	0.4946
reads the schema	144x	**0.6569**
the model writes the query	**1,408x**	**0.4043**

The first two were already on record for these exact questions. Only the third was missing, which is why the cost table had been mixing question sets. Graded 282 of 284; two answers came back ungradeable and both are among the smallest, not the largest.

> ### An unplanned cross-check, and it holds
>
> The held-back set gave this path **0.4039** five days ago. These development questions give **0.4043** — a difference of 0.0004, on a different question set, under a separately configured run on a different night.
>
> Nothing was arranged to produce that. It is the strongest evidence in this report that the measurement apparatus is sound, and it arrived by accident while filling a gap in a cost table.

No dollar figure is given anywhere here. The models run on hardware this project does not pay for per token, so seconds and tokens are the honest units and a dollar amount would be invented.

6b · The defect that made the first cost table wrong

The schema-reading path silently becomes a different system depending on one environment variable that nothing in the command line, the path's own definition, or the shared runner ever sets.

same 284 questions, changing only how long a call may take	default, 30 seconds	4,096 seconds
total time	**14.0 s**	**1,168.6 s**
successful model calls	**0**	**159**
tokens	0	817,238
answers carrying a fallback note	**198 / 284**	39 / 284

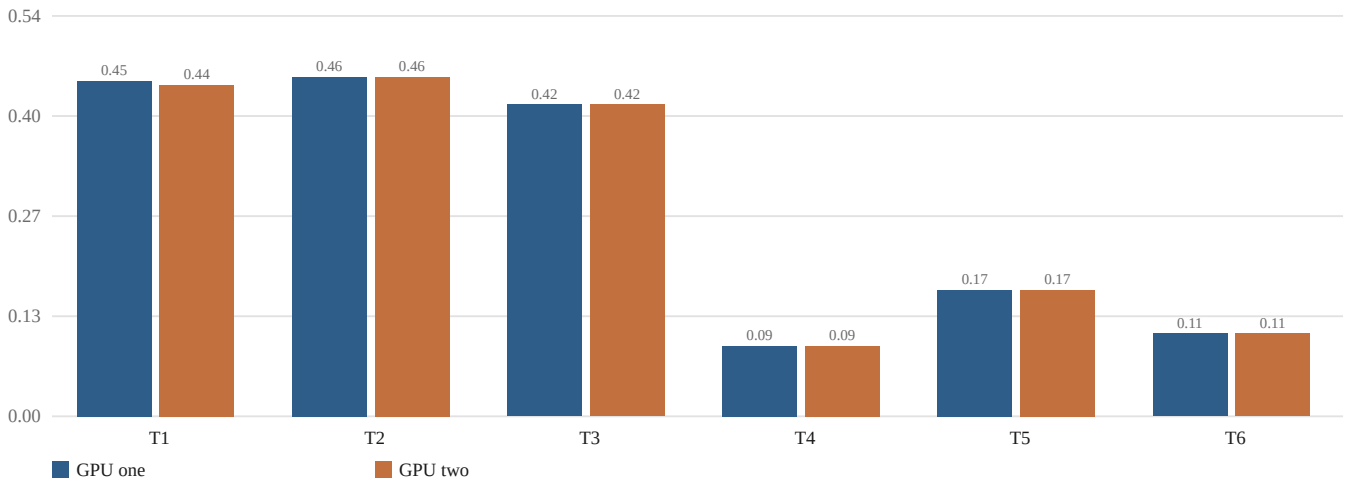
Eighty-three times faster, and not the same system. Every call times out, the pipeline quietly falls back to its model-free path, and finishes. No error appears anywhere. The only trace is a field on two answers in three that nobody reads unless they are looking for it.

This is the same failure class as the one in §5 — a plausible wrong number rather than a crash — but it lands on the path carrying the project's headline result rather than on its baseline. My own first cost table reported that path at 14 seconds and zero model calls, and it was wrong.

> ### The obvious worry, checked rather than assumed
>
> If that path can silently run as something else, were the published figures measured on the real thing? The check is a read of committed files, not a re-run: count how many saved answers carry the fallback note.
>
> | published run | answers | carrying a fallback note | rate |
> |---|---|---|---|
> | post-repair development | 3,407 | 122 | 0.04 |
> | holdout | 1,440 | 86 | 0.06 |
> | expansion | 852 | 19 | 0.02 |
>
> Against a broken reference of 0.70 and a healthy one of 0.14, all three sit at or below 0.06. **Their model seams were live. No published figure is affected**, and the headline comparison stands exactly as recorded.
>
> What remains is the fragility, not a number. The cheap guard is to make the runner refuse to start, or at least complain, when that rate goes high. That is a change in behaviour, so it was not made overnight beside a measurement.

7 · Does the result repeat?

The same arm was run 3 times on one GPU and 3 times on a second, with the cache cleared between runs so that what varies is the model and not a stored answer. Both instances serve identical weights, so this measures repeatability, not a comparison between models.



Questions answered with exactly the right records, by difficulty tier
Computed from 3 runs on one instance and 3 on the other

tier	GPU one mean	spread	GPU two mean	spread
T1	0.4497	0.0075	0.4444	0.0000
T2	0.4550	0.0075	0.4550	0.0075
T3	0.4183	0.0092	0.4183	0.0185
T4	0.0943	0.0000	0.0943	0.0000
T5	0.1698	0.0000	0.1698	0.0000
T6	0.1111	0.0000	0.1111	0.0000

7b · A third finding, carried in from yesterday and now verified

Earlier in the same working session, before tonight's runs, a diagnostic of the schema-reading path turned up something that had never been written down. It was recomputed from source tonight rather than trusted, and both figures reproduced exactly.

The CAPEC database's spec sheet lists 21 kinds of connection and gives safety rules for 16. The safety gate treats a connection missing from a partially-filled rule set as *forbidden*, and throws the query away before it ever runs:

```
pre-run gate rejected all drafts:
relationship 'HAS_ALTERNATE_TERM' has no allowed_edge_patterns in card
```

The system builds the correct query and is not allowed to ask it.

	on the 34 questions needing those five connections	on the other 250
reads the database's own structure	**0.0000**	0.6649
hand-written queries	0.2076	0.5452

Development half, 284 questions, recomputed 2026-09-10 from the post-repair scores.

An absolute zero, on the approach that leads everywhere else. And the asymmetry is worse than it looks: the rule exists only for one of the two databases, and the *other* database's spec sheet lists no safety rules at all — so it is unconstrained. A spec sheet that is 16-of-21 complete is punished where one that is 0-of-21 is not.

All five missing entries have exactly one possible answer in the data, so the repair is derivable rather than something anyone has to author. It would move the schema-reading path's development score from 0.585 to somewhere between 0.610 and 0.665.

> **Why this is not simply a bug to fix quietly**
>
> An earlier experiment repaired six defects in the **hand-written** approach before comparing the two, precisely so the comparison would be fair. This is the mirror-image defect, and repairing it widens a gap the project already reports in the schema-reading path's favour. It has to be published as that, not slipped in as a bug fix, or the comparison stops being trustworthy.

8 · What this means for the October pitch

Nothing in the deck has to change, and one thing in it gets stronger.

Do not change

The headline comparison between the three ways of answering is untouched. Tonight's work sits entirely inside a stage that ships switched off.

Gets stronger

The slide asking for the next round of funding currently says: *ask by name and it works, paraphrase the name and it scores zero — fund us to fix that.* That is a gap, not a plan. It can now say something better: we identified the weakest component, built the fix, measured it against a pre-registered rule, and the fix did not work — but the measurement told us why, and it was not what anyone expected. That is what a funded research programme looks like from the inside, and it is far more convincing to an engineer than another round of numbers that only go up.

The thing to be careful about

Both defects found tonight were **measurement** defects, not system defects, and both would have produced confident, plausible, wrong numbers. If anyone in that room reads this repository, the fact that these were caught and written down — rather than quietly fixed — is the strongest evidence of rigour the project has. It should be said out loud rather than hidden.

9 · What was recorded, and what is still open

Everything below is committed. Nothing was pushed.

record	what it holds
`EXP-2026-09-015`	the pre-registered protocol, the null result, and both measurement defects
`EXP-2026-09-016`	the cost and latency protocol, the drawn sample, and the silent-empty-query defect
`Q44`	the open decision: repair the answer key, or teach the scorer to accept what the database has
`scripts/rescore_slot_relation.py`	reports both readings of the slot scores and edits nothing
token metering	every model call's cost is now read off the wire rather than estimated

The decision waiting for you

The eleven bad answer-key records can be repaired from the database, or the scorer can be taught to accept any connection the database declares between the two kinds of thing the key names. The first is more honest and invalidates two experiments' slot figures; the second is cheaper and leaves a known-wrong answer key in place. Repairing an answer key after seeing a score is exactly the move that evaluation discipline exists to prevent, which is why it was left to you.

What was not done

The held-back half of the question bank was not touched, and neither was the sealed holdout. No shipped behaviour changed. The flags that control the rewrite stage remain off, as the previous experiment left them.